

Types of variable:-

- 1)Value
- 2)Pointer
- 3)Reference

1)Value:- A value variable stores the actual value of a data type. When we assign or pass a value variable to another variable or function, a copy of the value is made.

Memory: The value variable directly stores the data in memory.

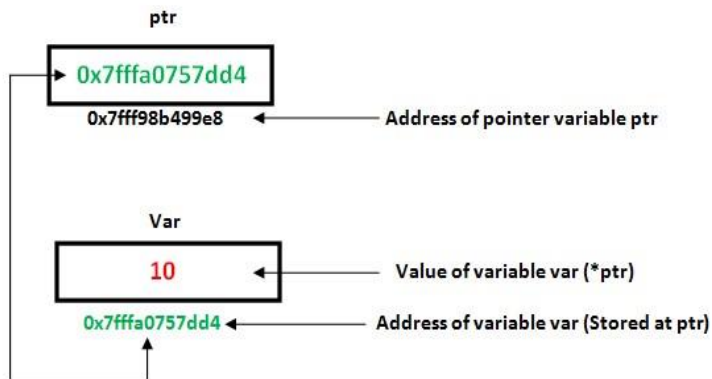
- **int** - stores integers (whole numbers), without decimals, such as 123 or -123.
- **double** - stores floating point numbers, with decimals, such as 19.99 or 19.99.
- **char** - stores single characters, such as 'a' or 'B'. ...
- **string** - stores text, such as "Hello World". ...
- **bool** - stores values with two states: true or false.

2)Pointer:-

A pointer is a variable that holds the memory address of another variable. A pointer needs to be dereferenced with the * operator to access the memory location it points to.

Like regular variables, pointers have a data type. For example, a pointer of type integer can hold the address of a variable of type integer. A pointer of character type can hold the address of a variable of character type. You should see a pointer as a symbolic representation of a memory address.

Following fig. shows pointer:-



Syntax:-

```
datatype *var_name;
```

```
int *ptr;    //ptr can point to an address which holds int data
```

Example –

```
#include <iostream.h>  #include<conio.h>

void main() {

int number=30;

int * p;

p=&number;    //stores the address of number variable    cout<<"Address of
number variable is:"<<&number<<endl;    cout<<"Address of p variable
is:"<<p<<endl;    cout<<"Value of p variable is:"<<*p<<endl;

getch();

}
```

Output:-

Address of number variable is:0x7ffccc8724c4

Address of p variable is:0x7ffccc8724c4

Value of p variable is:30

3)Reference:-

When a variable is declared as a reference, it becomes an alternative name for an existing variable.

Reference can be created by simply using an ampersand (&) operator. When we create a variable, then it occupies some memory location. We can create a reference of the variable; therefore, we can access the original variable by using either name of the variable or reference. For example, `int a=10;` Now, we create the reference variable of the above variable.

```
int &ref=a;
```

The above statement means that 'ref' is a reference variable of 'a', i.e., we can use the 'ref' variable in place of 'a' variable.

Example:-

```
#include
<iostream.h>
#include<conio.h>
int main() {

int x = 10;

// ref is a reference to x.

    int & ref = x;

        // Value of x is now changed to
20    ref = 20;

cout << "x = " << x << '\n';

    // Value of x is now changed to 30

    x = 30;

cout << "ref = " << ref << '\n';

getch(); }
```

Output:

```
x = 20  ref
= 30
```

Functions in C++ :

A function is a set of statements that take inputs, do some specific computation, and produce output.

OR

“Function is self contained block of statements or instructions, after execution of such statements it produces desired out put such a block of statements is called as **FUNCTION**”

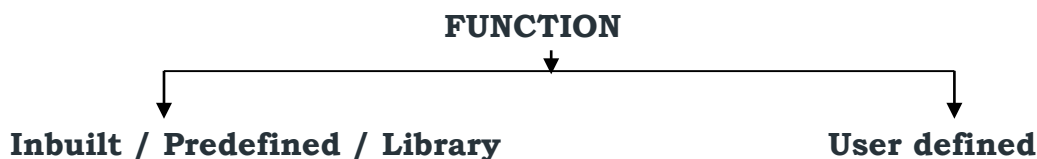
You need to give a particular name to the function and write some logic or group of information inside it. And then you can invoke that function from the main function. For example, you can name a function as factorial, and inside that function, you can write the logic to find the factorial of a number. And then, you can use it whenever you need that function in the program.

Syntax :-

```
Return_Type Function_name(Parameter1, parameter2,.....)
{
    //function body
}
```

E.g:- `int add ( int a, int b )`
{
 //function body
}

Mainly function can be categorized into two parts which is shown in following figure:



1).Built-in Functions/ Predefined /Library function:-

“Inbuilt functions are those functions which were all ready defined under particular header file such functions are called as Inbuilt or predefined or library functions”

For eg:- `log()`,`sqrt()`,`pow()` etc.

2).User Defined Function:-

User Defined functions are user/customer-defined blocks of code specially customized to reduce the complexity of big programs.

OR

“The functions which are defined by user is called as user defined functions”

C++ also allows its user to define their own functions. We can define the function anywhere in program and then call these function from main() function.

Advantages or need of function:

- 1)Function allows easy to understand, debug and testing the program.
- 2)Due to function execution of program increases.
- 3)The length of program can be reduced by using function at suitable place

Steps to add user defined function into program:

If we have to add our own defined (user defined) function into program at that time we must have to do following three important things:

1) Function Declaration (Function Prototyping):

First we have to declare the function and is declared as follows:

Syntax: `return_type funct_Name(argument list);`

here:

`return_type` : is value return from the function which is any valid data type in C++ language

`funct_Name` : is an identifier (name given by programmer)

`argument list` : is value accept by the function and also every argument separated from each other by comma (,)

Also, generally functions are declared in global section of program

Note:

In function declaration `return_type` is optional part. If we do not specify `return_type` of function then by default that function return integer value. e.g.:

1) `float    get( int, int );`

here, `get` is a function which accepts two integer type arguments and it returns one float type value.

2) Function Calling:

After declaration of function we must have to call the declared function and is called as fallow:

Syntax:

`funct_Name(arg1, arg2, . . . . . , argN);`

here;

funcName is declared function name arg1,
arg2, ,argN are values accept by the function.

Also, generally functions are called from main() e.g.:

The above declared function can be called as fallows:

```
int x , y ;  
x= 50; y=100;  
get( x , y ); // function calling
```

3) Function Definition (Function Implementation):

Function can be defined by following way:

Syntax:

```
return_type funcName(datatype arg1,datatype arg2, . . . ,datatype argN)  
{  
    Body of function  
}
```

Also, generally functions are defined outside the main() e.g.:

The above declared function can be defined as fallows:

```
float get( int p, int q)  
{  
    Body of get( ) function  
}
```

Types Of Arguments:

Actual Parameters (Actual Arguments):

“The variables which are used at function call are known as Actual Parameters or Actual Arguments”

Formal Parameters (Formal Arguments):

“The variables which are used at function definition are called as Formal Parameters or Formal Arguments”

Scope of variable:

Local Variable:

“The variables which are declared inside the body of particular function are called as Local Variables”

The scope of local variable is limited within the function body i.e. local variable can not accessed inside another function body

Global Variable:

“The variables which are declared under global section of program are called as Global Variables”

The scope of global variable is unlimited i.e. global variables are easily accessed within every function of program.

Types of User defined Function:

Depending upon return type and arguments accept by the function, the function can be classified into four types and they are as follows:

- 1) Function with argument without return value.
- 2) Function with argument with return value.
- 3) Function without argument with return value.
- 4) Function without argument without return value.

```
#include<iostream.h>
#include<conio.h>
void funtion1(); // Function Declaration - no argument and no return value
int funtion2(); // Function Declaration - no argument but return value
void funtion3(int a, int b); // Function Declaration - argument but no return value
int funtion4(int a, int b); // Function Declaration - argument but return value
void main()
{
    int x, y;
    clrscr();
    cout << "Simple Function Types Example Program in C++\n"; // Function
    Call - no argument and no return value
    funtion1();
    // Function Call - no argument but return value
    = funtion2();
    cout << "\nReturn Value : " << x;
    // Function Call - no argument and no return value
    funtion3(10, 20);
    // Function Call - no argument but return value
    = funtion4(100, 200);
    cout << "\nAddition : " << y;
    getch();
}
```

```

}
void funtion1()          // Function Definition - no argument and no return value
{
    cout << "\nIts simple function definition";
}
int funtion2()           // Function Definition - no argument but return value
{
    int c = 1000;
    return c;
}

void funtion3(int a, int b) // Function Definition - argument but no return
value
{
    int c;
    c = a + b;
    cout << "\nAddition : " << c;
}
int funtion4(int a, int b) // Function Definition - argument but return value
{ int c;
    c = a + b;
    return c;
}

```

Output:-

Its simple function definition

Return Value : 1000

Addition : 30

Addition : 300

Default arguments:-

Default arguments:-

A default argument is a value in the function declaration automatically assigned by the compiler if the calling function does not pass any value to that argument.

Advantages of Default Arguments:-

1.Default arguments are useful when we want to increase the capabilities of an existing function as we can do it just by adding another default argument to the function.

2. It helps in reducing the size of a program.
3. It provides a simple and effective programming approach.
4. Default arguments improve the consistency of a program.

Disadvantages of Default Arguments:-

It increases the execution time as the compiler needs to replace the omitted arguments by their default values in the function call.

Eg:-

```
#include<iostream.h>
#include<conio.h>
int sum(int x, int y, int z=0, int w=0)//Here there are two values in the default
arguments
{
// Both z and w are initialised to zero
return (x + y + z + w); // return sum of all parameter values
}
void main()
{
clrscr();
cout << sum(10, 15) << endl; // x = 10, y = 15, z = 0, w = 0
cout << sum(10, 15, 25) << endl; // x = 10, y = 15, z = 25, w = 0
cout << sum(10, 15, 25, 30) << endl; // x = 10, y = 15, z = 25, w = 30
getch();
}
```

Output:-

```
25
50
80
```

Parameter passing techniques:

C++ language supports three kinds of parameter passing techniques;

- 1) Pass by Value (Call by value)
- 2) Pass by Pointer (Call by pointer) or Pass by address
- 3) Pass by reference.

Let us see all these techniques in details:

1) Pass by Value (Call by value):

Passing value of variable to called function is called as “Pass by value” or “call by value”

In this case only value of actual parameter is copied into formal parameter therefore making some operations on formal parameter does not affect on value of actual parameter.

Syntax for calling:

Funct_name(variable_name);

Eg:-

```
#include <iostream.h>
#include<conio.h>
void fun(int a) {
    a=20;
}
void main()
{
    int a =30;
    clrscr();
    fun(a);
    cout<<"Value of A: "<<a<<endl;
    getch();
}
```

Output:- value of A: 30

2) Pass by Pointer (Call by pointer/Address):

Passing address of variable to called function is called as “Pass by pointer/address” or “call by pointer/address”

In this case address of actual parameter is copied into formal parameter therefore making some operations on formal parameter affects on value of actual parameter.

Syntax for calling:

Funct_name(&variable_name);

OR

Funct_name(pointer);

Eg:-

```
#include <iostream.h>
#include<conio.h>
```

```
void fun(int *b)
{
    *b=20;
}
void main()
{
    clrscr();
    int a =10;

    fun(&a);

    cout<<"Value of A: "<<a<<endl;    getch();
}
```

Output:- value of A: 20

3) Pass by Reference (Call by reference):

Passing reference variable (alias) of existing variable to called function is called as “Pass by reference” or “call by reference”

In this case alias of actual parameter is copied into formal parameter therefore making some operations on formal parameter affects on value of actual parameter.

Syntax for calling:

Funct_name(Reference_var_name);

OR

Funct_name(variable_name);

Eg:-

```
#include <iostream.h>
```

```
#include<conio.h>
```

```
void fun(int &b)
```

```
{
```

```
    b=20;
```

```
}
```

```
void main()
```

```
{
```

```
    clrscr();
```

```
    int a =10;
```

```
    fun(a);
```

```
    cout<<"Value of A: "<<a<<endl;
```

```
    getch();
```

```
}
```

Output:- value of A: 20